

FATEC SP

Campus: Zona Leste

Curso: Desenvolvimento de Software Multiplataforma

Disciplina: Desenvolvimento de Web 3

Professora: Luciana Akemi

Aluno: Adriano Rodrigues de Araujo

Atividades: roteiro de projeto condomínio no vscode

1. Preparação do Ambiente e Pastas

No VS Code, crie uma pasta raiz chamada `sistema-condominio`. Dentro dela, abra o terminal e organize a estrutura conforme as boas práticas pedagógicas de sistemas técnicos:

1. **Criação das pastas:**
 - o `src/database`: Scripts SQL.
 - o `src/controllers`: Lógica de sorteio e cadastros.
 - o `src/routes`: Caminhos da API.
 - o `tests/`: Arquivos de teste.
2. **Inicialização do Node.js:** No terminal, digite `npm init -y`.
3. **Instalação das dependências:**

Bash

```
npm install express mysql2 cors dotenv  
npm install --save-dev jest supertest
```

(O Jest será nossa ferramenta de teste para validar o sorteio das garagens.)

2. Banco de Dados (MySQL)

Crie o arquivo `src/database/schema.sql`. Note que adicionamos flags para identificar quem **não** entra no sorteio:

SQL

```
CREATE TABLE garagens (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  numero_vaga INT UNIQUE,  
  eh_fixa BOOLEAN DEFAULT FALSE, -- TRUE para Síndico/Subsíndico  
  ocupada BOOLEAN DEFAULT FALSE  
);  
  
CREATE TABLE condominos (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  nome VARCHAR(100),  
  qtd_veiculos INT DEFAULT 1, -- Se for 1, não vai para o sorteio  
  cargo VARCHAR(50) DEFAULT 'Morador'  
);
```

3. Lógica do Sorteio e Regras de Negócio

No arquivo `src/controllers/GaragemController.js`, implementamos a lógica que filtra as exceções solicitadas:

JavaScript

```
// Exemplo da lógica de filtro para o sorteio  
const realizarSorteio = (vagasLivres, moradoresSorteio) => {  
  // Regra: Apenas moradores com > 1 veículo e vagas não fixas participam  
  const vagasParaSortear = vagasLivres.filter(v => !v.eh_fixa);  
  const moradoresValidos = moradoresSorteio.filter(m =>  
    m.qtd_veiculos > 1 && m.cargo === 'Morador');  
  
  // Embaralhamento (Shuffle)  
  return moradoresValidos.map((morador, index) => ({  
    morador: morador.nome,  
    vaga: vagasParaSortear[index]  
  }));  
};
```

4. Como Incluir e Rodar os Testes

Para garantir que o síndico não entre no sorteio por erro, criamos testes automatizados no arquivo `tests/sorteio.test.js`.

O que testar?

1. **Teste de Exclusão:** Verificar se vagas marcadas como `eh_fixa` não aparecem no resultado do sorteio.
2. **Teste de Quantidade:** Verificar se moradores com apenas 1 carro foram ignorados pela função.
3. **Teste de Cadastro:** Validar se o utensílio da cozinha foi inserido com nome e quantidade correta.

Exemplo de Código de Teste (Jest):

Roteiro2:

```
test('Deve ignorar o síndico no sorteio de vagas comuns', () => {
  const moradores = [
    { nome: 'João (Síndico)', cargo: 'Síndico', qtd_veiculos: 2 },
    { nome: 'Maria', cargo: 'Morador', qtd_veiculos: 2 }
  ];
  const resultado = realizarSorteio(vagas, moradores);
  // Verifica se o Síndico NÃO está no array de resultado
  expect(resultado).not.toContainEqual(expect.objectContaining({
    morador: 'João (Síndico)' })));
});
```

Para rodar os testes: No terminal, digite `npx jest`.

5. Gerenciamento de Documentos e Chat

- **Chat:** Utilize o banco de dados para salvar as mensagens com ID do condômino e texto. No Frontend, use `setInterval` ou `WebSockets` para atualizar a tela de conversas.
 - **Documentos (PDF):** Para a relação de utensílios, você pode usar a biblioteca `jsPDF` no JavaScript. Ela lê os dados do banco (ex: "Garfo: 20 unidades") e gera um arquivo pronto para impressão.
-

Resumo da Execução no VS Code:

1. Configure o arquivo `.env` com as credenciais do seu MySQL.
2. Crie as rotas em `src/routes/` para conectar o Frontend (HTML) ao Banco de Dados.
3. Utilize o **Thunder Client** (extensão do VS Code) para testar se os cadastros de condôminos e utensílios estão salvando no banco antes mesmo de criar o HTML.

B-Atividade2:roteiro

1. Organização de Pastas (Estrutura do Projeto)

Uma boa organização é fundamental para que o código seja escalável e fácil de manter. Utilizaremos o padrão **MVC** (Model-View-Controller) simplificado.

Plaintext

```
condo-check-pro/
├── backend/                # Lógica do servidor (Node.js)
│   ├── config/            # Conexão com o Banco de Dados
│   ├── controllers/       # Lógica de sorteio e regras de negócio
│   ├── routes/            # Definição das rotas da API
│   └── server.js          # Arquivo principal do servidor
├── frontend/              # Interface do usuário (HTML/CSS/JS)
│   ├── public/            # Imagens e ícones
│   ├── css/               # Estilização (Bootstrap)
│   ├── js/                # Scripts de interação e sorteio
│   └── index.html         # Dashboard principal
├── docs/                  # PDFs gerados e documentos do salão
└── sql/                   # Scripts de criação do banco de dados
```

2. Passo a Passo do Banco de Dados (MySQL)

O banco de dados precisa "entender" quem é síndico e quem tem vaga fixa para não incluí-los no sorteio automático.

1. **Tabela de Unidades:** Registra o apartamento e se a vaga dele é fixa por direito (ex: contrato ou cargo).
 2. **Tabela de Condôminos:** Armazena o perfil. Se o campo `qtd_veiculos` for igual a 1, a regra de negócio do sorteio irá ignorar este morador.
 3. **Tabela de Garagens:** Crucial ter um campo `eh_fixa` (booleano). Vagas de síndico/subsíndico são marcadas como 1 (true) para nunca entrarem no bolo do sorteio.
-

3. Lógica Detalhada do Sorteio (JavaScript)

O sorteio não é apenas um "random". Ele segue este fluxo lógico no backend:

- **Passo 1 (Filtro):** O sistema busca no banco apenas as garagens onde `eh_fixa = 0`.
 - **Passo 2 (Filtro):** O sistema busca moradores que não são síndicos e que possuem mais de 1 veículo.
 - **Passo 3 (Embaralhamento):** Utilizamos o algoritmo de *Fisher-Yates* para garantir que a aleatoriedade seja real e não viciada.
 - **Passo 4 (Atribuição):** O script vincula o ID da unidade ao ID da vaga sorteada e atualiza o banco de dados.
-

4. Gerenciamento de Chat e Documentos

Para que o sistema seja funcional, ele utiliza JavaScript para duas funções distintas:

O Chat (Grupo de Comunicação)

- As mensagens são enviadas via **POST** para o servidor.
- O banco de dados armazena a mensagem com um *timestamp* (data/hora).
- O frontend faz um "refresh" ou usa WebSockets para exibir as novas mensagens dos vizinhos em tempo real.

Geração de Documentos (Reserva e Utensílios)

- **Reserva:** O morador escolhe a data; o JavaScript verifica se já existe uma reserva no banco. Se livre, gera um comprovante em tela.
- **Utensílios:** O sistema lê a tabela de `utensilios_cozinha` e monta uma tabela HTML dinâmica. O usuário pode clicar em "Imprimir", e o CSS usa `@media print` para formatar o documento pronto para o papel.

5. Como rodar o projeto

1. **Banco:** Execute o script SQL na sua ferramenta (MySQL Workbench ou PHPMyAdmin).
2. **Dependências:** Dentro da pasta `backend`, rode `npm install express mysql2 cors`.
3. **Servidor:** Inicie o servidor com `node server.js`.
4. **Frontend:** Abra o `index.html` no navegador. Ele fará as requisições para o backend via porta 3000.

Essa estrutura garante que as regras solicitadas (vagas fixas, sorteio aleatório e chat) funcionem de forma integrada e organizada

C-Apresento a estrutura completa para o sistema de "CondoCheck".

1. Banco de Dados (MySQL)

O esquema armazena os dados dos moradores e o registro de entradas/saídas.

SQL

```
CREATE DATABASE condominio_db;
USE condominio_db;

CREATE TABLE acessos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome_morador VARCHAR(100) NOT NULL,
  apartamento VARCHAR(10) NOT NULL,
  tipo_acesso ENUM('Entrada', 'Saída') NOT NULL,
  data_hora DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

2. Backend (Node.js + Express)

Este código gerencia a conexão com o banco e fornece a API para o monitoramento.

JavaScript

```
// server.js
const express = require('express');
const mysql = require('mysql2');
const cors = require('cors');
const app = express();

app.use(express.json());
app.use(cors());

const db = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: 'password',
  database: 'condominio_db'
});

// Rota para listar acessos
app.get('/api/aceessos', (req, res) => {
  db.query('SELECT * FROM acessos ORDER BY data_hora DESC', (err,
  results) => {
    if (err) return res.status(500).send(err);
    res.json(results);
  });
});

// Rota para registrar novo acesso
app.post('/api/aceessos', (req, res) => {
  const { nome, ap, tipo } = req.body;
  const sql = 'INSERT INTO acessos (nome_morador, apartamento,
  tipo_acesso) VALUES (?, ?, ?)';
  db.query(sql, [nome, ap, tipo], (err, result) => {
    if (err) return res.status(500).send(err);
    res.json({ message: 'Acesso registrado!' });
  });
});

app.listen(3000, () => console.log('Servidor rodando na porta 3000'));
```

3. Frontend Responsivo (HTML + CSS + JS)

HTML (index.html)

Utiliza componentes do Bootstrap para garantir a responsividade.

+1

HTML

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
  scale=1">
  <title>CondoCheck - Monitoramento</title>
```

```

    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.
min.css" rel="stylesheet">
    <link rel="stylesheet" href="style.css">
</head>
<body class="bg-light">
    <nav class="navbar navbar-dark bg-primary shadow">
        <div class="container">
            <span class="navbar-brand mb-0 h1">CondoCheck
Monitor</span>
        </div>
    </nav>

    <div class="container mt-4">
        <div class="row">
            <!-- Formulário de Registro -->
            <div class="col-md-4">
                <div class="card p-3 shadow-sm">
                    <h5>Registrar Acesso</h5>
                    <form id="acessoForm">
                        <input type="text" id="nome" class="form-
control mb-2" placeholder="Nome do Morador" required>
                        <input type="text" id="ap" class="form-control
mb-2" placeholder="Apartamento" required>
                        <select id="tipo" class="form-select mb-3">
                            <option value="Entrada">Entrada</option>
                            <option value="Saída">Saída</option>
                        </select>
                        <button type="submit" class="btn btn-primary
w-100">Registrar</button>
                    </form>
                </div>
            </div>

            <!-- Tabela de Monitoramento Real-time -->
            <div class="col-md-8">
                <div class="card p-3 shadow-sm">
                    <h5>Últimos Acessos</h5>
                    <div class="table-responsive">
                        <table class="table table-striped mt-2">
                            <thead>
                                <tr>
                                    <th>Morador</th>
                                    <th>Apto</th>
                                    <th>Tipo</th>
                                    <th>Horário</th>
                                </tr>
                            </thead>
                            <tbody id="tabelaAcessos">
                                <!-- Dados via JS -->
                            </tbody>
                        </table>
                    </div>
                </div>
            </div>
        </div>
    </div>

    <script src="app.js"></script>
</body>
</html>

```

JavaScript Frontend (app.js)

Gerencia a interação com a API.

JavaScript

```
document.getElementById('acessoForm').addEventListener('submit', async
(e) => {
    e.preventDefault();
    const dados = {
        nome: document.getElementById('nome').value,
        ap: document.getElementById('ap').value,
        tipo: document.getElementById('tipo').value
    };

    await fetch('http://localhost:3000/api/acessos', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(dados)
    });

    carregarAcessos();
    e.target.reset();
});

async function carregarAcessos() {
    const res = await fetch('http://localhost:3000/api/acessos');
    const acessos = await res.json();
    const corpoTabela = document.getElementById('tabelaAcessos');

    corpoTabela.innerHTML = acessos.map(a => `
        <tr>
            <td>${a.nome_morador}</td>
            <td>${a.apartamento}</td>
            <td><span class="badge ${a.tipo_acesso === 'Entrada' ?
'bg-success' : 'bg-danger'}">${a.tipo_acesso}</span></td>
            <td>${new Date(a.data_hora).toLocaleString('pt-BR')}</td>
        </tr>
    `).join('');
}

// Inicializa a tabela ao carregar
carregarAcessos();
```

Explicação Técnica:

- **Box Model e Responsividade:** O layout utiliza o sistema de grid do Bootstrap (`col-md-4`, `col-md-8`) para garantir que o formulário fique acima da tabela em dispositivos móveis e ao lado em telas maiores.
- **Interatividade:** O JavaScript (Node.js no backend e Fetch API no frontend) substitui as ações estáticas para criar uma aplicação dinâmica.
- **Estilização:** As bordas e sombras (`shadow-sm`) foram aplicadas seguindo os conceitos de propriedades de CSS para organização visual de "containers".